LECTURE NOTES

Lecture 6 - Gradient-based Learning & Backpropagation

AI506 — Advanced Machine Learning

Simon Holm

March, 2026

SDU

DEPARTMENT OF MATHEMATICS
AND COMPUTER SCIENCE

Contents

The Central Idea

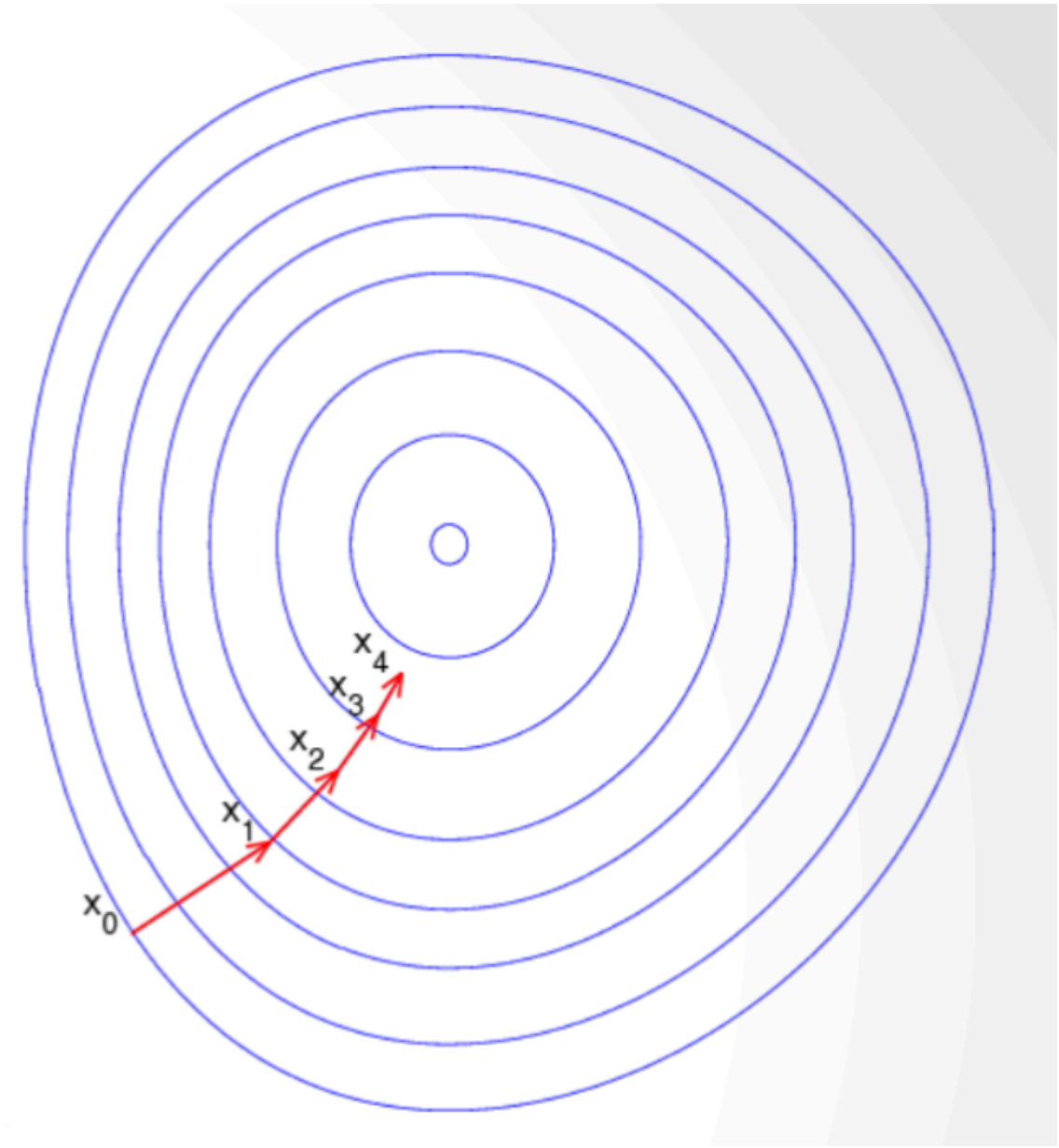

Figure 1: Updating the model parameters following the steepest slope

The derivative specifies how to scale a small change in input to obtain a corresponding change in the output:

$$f(x + \varepsilon) \approx f(x) = \varepsilon f'(x)$$

We then know that $f(x - \varepsilon \text{ sign}(f'(x))) < f(x)$

For this we need partial derivatives

$$\frac{\partial}{\partial x}(x_i) f(x)$$

Measures how f changes as only variable x_i increases at point x

Then the gradient contains all the prtial derivatives: $\nabla_x f(x)$

Then $x' = x - \varepsilon \nabla_x f(x)$ "with "ε" bein the learning rate"

Stochastic Gradient Descent (SGD)

"Stochastic" graident is estimated.

In each step of SGD we can sample a minibatch of examples

$$B = \{x_1, ...x_{m'}\}$$

- drawn uniformly from the training set
- Minibatch size m' is typically chosen to be small: 1 to a hundred
- Crucially m' is held fixed even if sample set is in billions
- We may fit a training set with billions of examples using updates computed on only a hundred examples
- This is mostly due to comutational limits (practical)
- This also includes randomness which can generally be good

SGD Estimate using minibatch

We can estimate the gradient by:

$$g = \frac{1}{m'} \nabla_\theta \sum_{i=1}^{m'} L(x_i, y_i, \theta)$$

Using **only** the examples of the minibatch

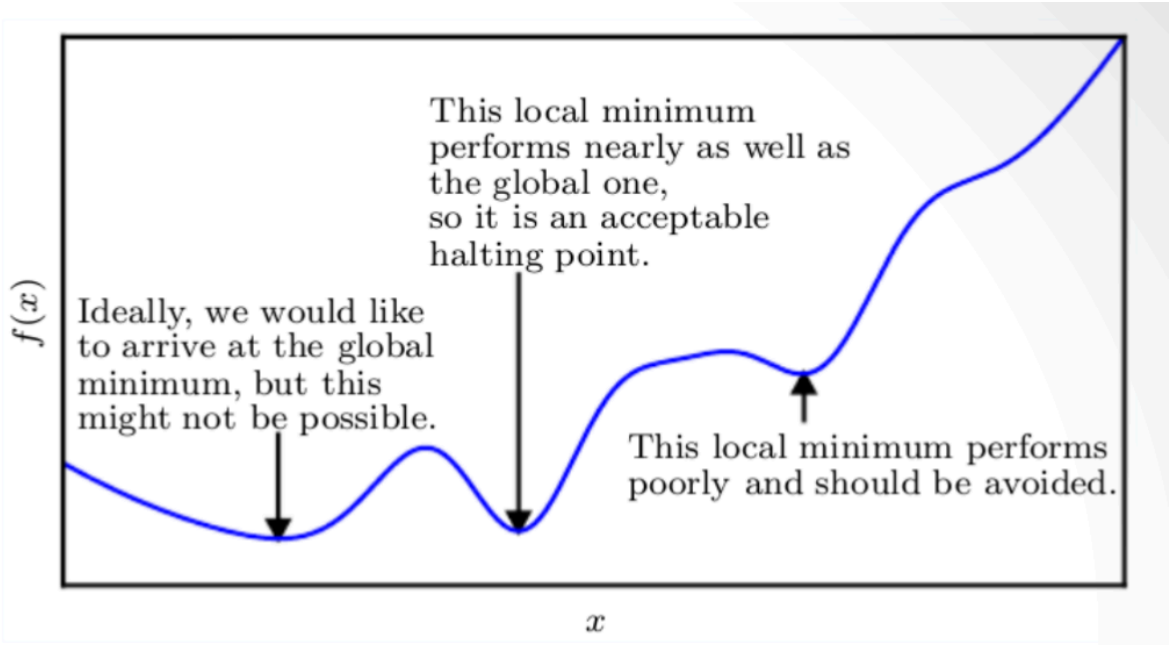

Figure 2: The Gradient Descent is not perfect, but good enough in practice

Specialties

Neural Network training not different from ML models with gradient descent. The components are needed:

1. optimization procedure, e.g., gradient descent
2. cost function, e.g., MLE
3. model family, e.g., linear with basis functions

The difference lies in the fact that nonlinearity causes non-convex loss

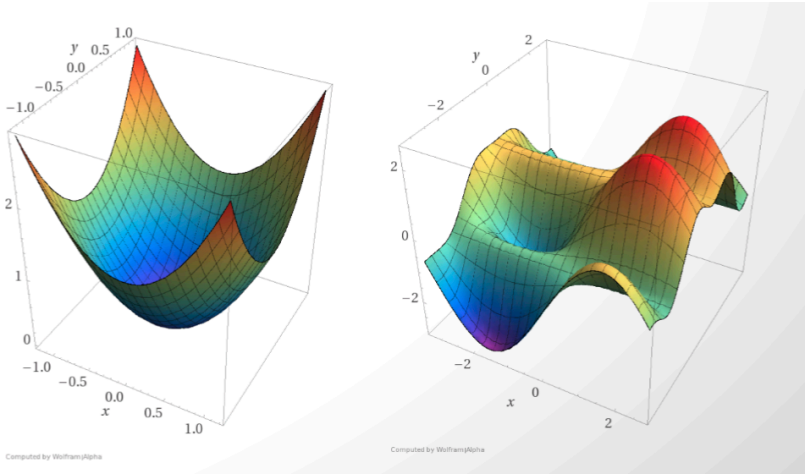

Figure 3: Convex vs. Non-Convex functions

Problems

This introduces a number problems..

1. We can end-up in local minima

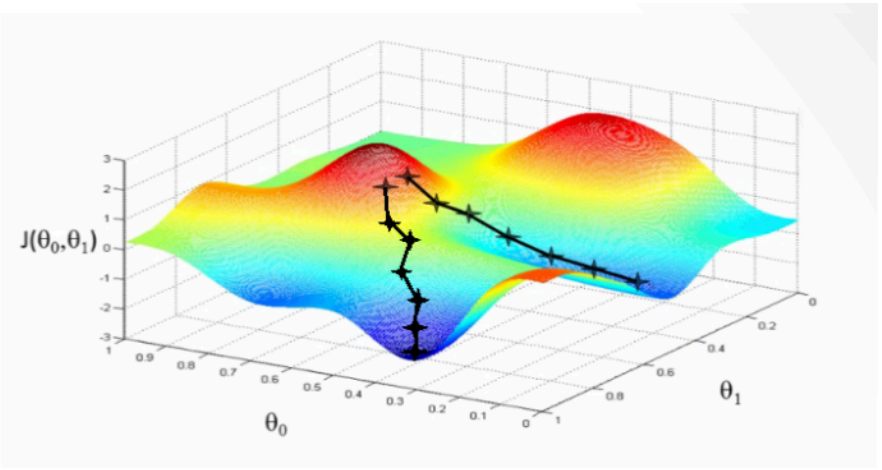

Figure 4: Example of an example finding a local minima (which is not necessarily the minimizer)

2. Stationary points

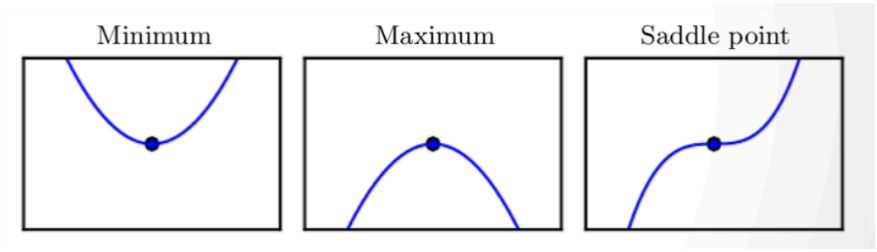

Figure 5: Saddle Points where $f'(x) = 0$ are neither maxima nor minima

3. Cliffs and Exploding Gradients

Neural networks with many layers have steep regions i.e., cliffs. Gradient update step can move parameters extremely far, jumping off cliff altogether.

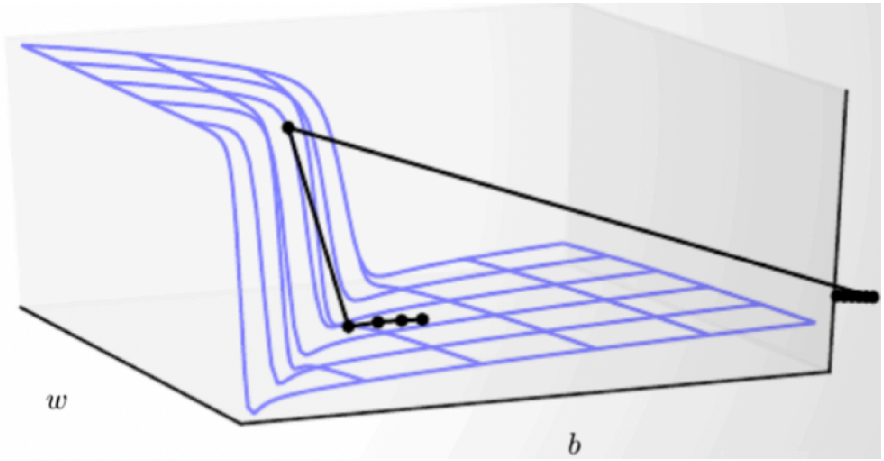

Figure 6: Example of a dangerous cliff

4. Inexact Gradients

Optimization algorithms assume we have access to exact gradient or Hessian matrix. In practice we have a noisy or biased estimate (e.g. using minibatch)

5. Bad initial points

Optimization based on local downhill moves can fail if local surface does not point towards the global solution

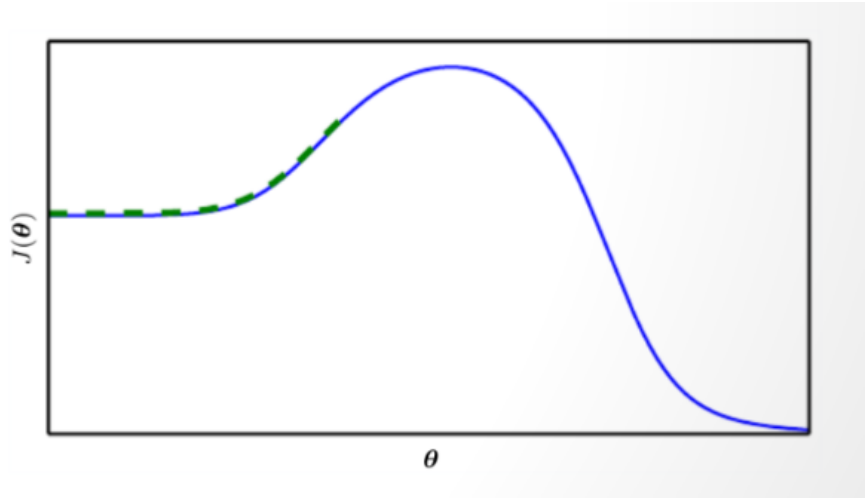

Figure 7: We need a good inital point to find a good minimum

5. The Learning Rate

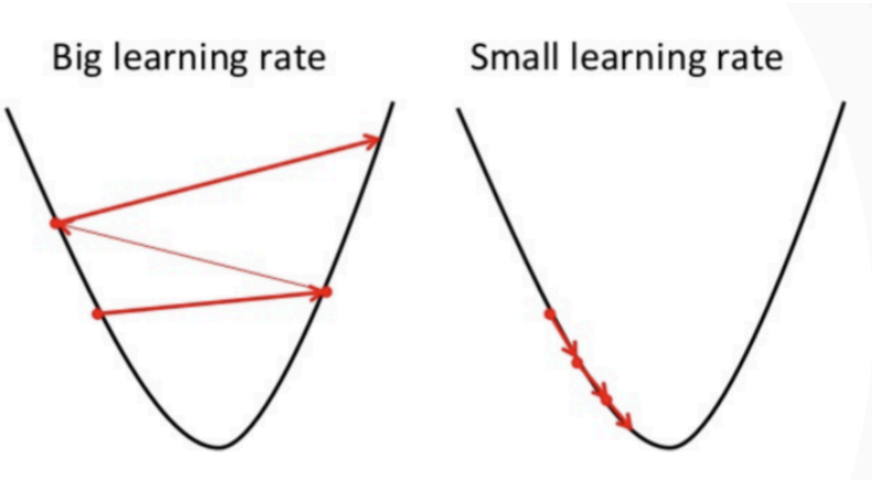

Figure 8: Loss might not converge when learning rate is too big

Reduce learning rate if no convergence.

Backpropagation

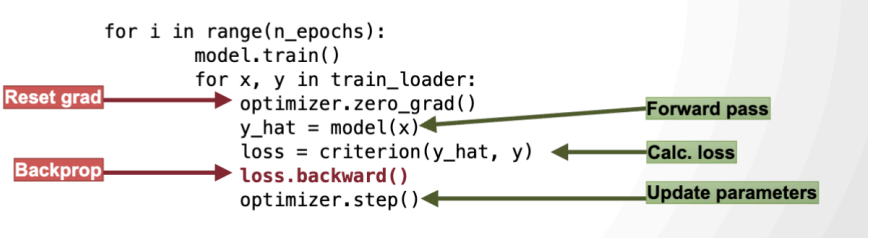

Figure 9: example of backpropagation usage

We use the chainrule. In Leibnitz notation

if $y = f(u)$ and $u = g(x)$ then

$$\frac{\mathrm{d}}{\mathrm{d}x}y = \frac{\mathrm{d}}{\mathrm{d}u}y \cdot \frac{\mathrm{d}}{\mathrm{d}x}u$$

Decomposing a neural net into functions

Consider $f(x) = W_2\ \mathrm{ReLU}(W_1 x)$ representing a 2-layer NN.

This can be composed into:

$$a(x) = W_1 x, \quad z(x) = \mathrm{ReLU}(x), \quad y(x) = W_2 x$$

Then

$$f(x) y(z(a(x)))$$

Then we have the Loss $E(f(x), y)$

Differentiating the individual functions

Take a unitds weight $a(z) \boldsymbol{w}^T \boldsymbol{z} = w_1 z_1, ... w_n z_n$

then

$$\frac{\mathrm{d}}{\mathrm{d}w_i} a = z_1, \quad \frac{\mathrm{d}}{\mathrm{d}z_i} a = w_1$$

Take the leaniar unit $\mathrm{ReLU}(x) = \max(0, x)$

$$\frac{\mathrm{d}}{\mathrm{d}x'} = 1 \text{ if } x > 0 \text{ and } 0 \text{ otherwise}$$

Loss function example $E(f(x), y) = \frac{1}{2}(f(x) - y)^2$

Then

$$\frac{\mathrm{d}}{\mathrm{d}f(x)} = f(x) - y$$

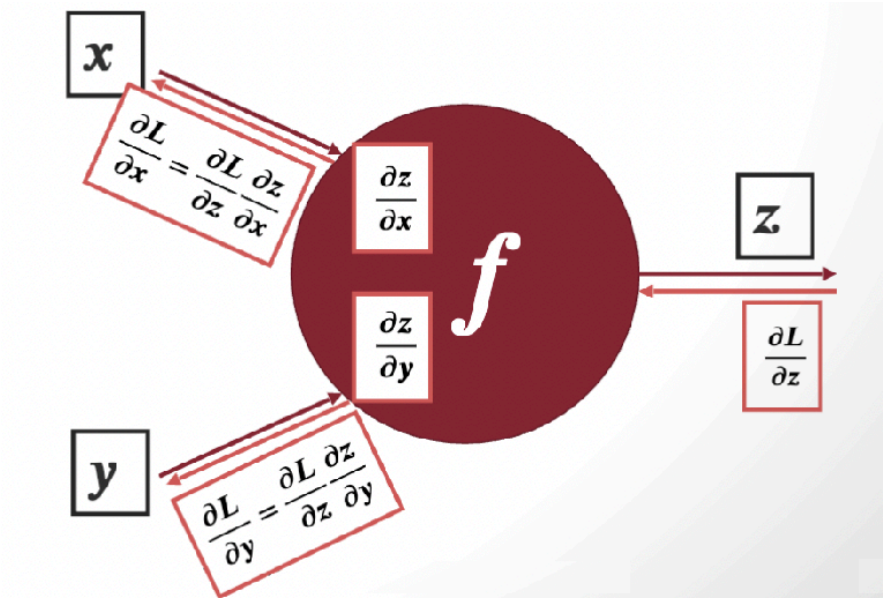

Figure 10: Backpropagation: A Simple Example

General feedforward networks

Given that $a : j = \sum_i w_{ji} z_i$

With an activation function $z_j = a_j$

Then $\frac{\partial}{\partial w_{ji}} E_n = \frac{\partial}{\partial a_j} E_n \cdot \frac{\partial}{\partial w_{ji}} a_j$

Usually we give the notation that the 'errors': $\delta_j = \frac{\partial}{\partial a_j} E_n$

so

$$\frac{\partial}{\partial w_{ji}} E_n = \delta_j z_i$$

We can then backpropagate, each node to learn the δ_i for each node

So for each layer

$$\delta_j = \underbrace{h'(a_j)}_{\substack{\text{Derivative of} \\ \text{activation function}}} \cdot \sum_k w_{kj} \overbrace{\delta_k}^{\text{output error}}$$

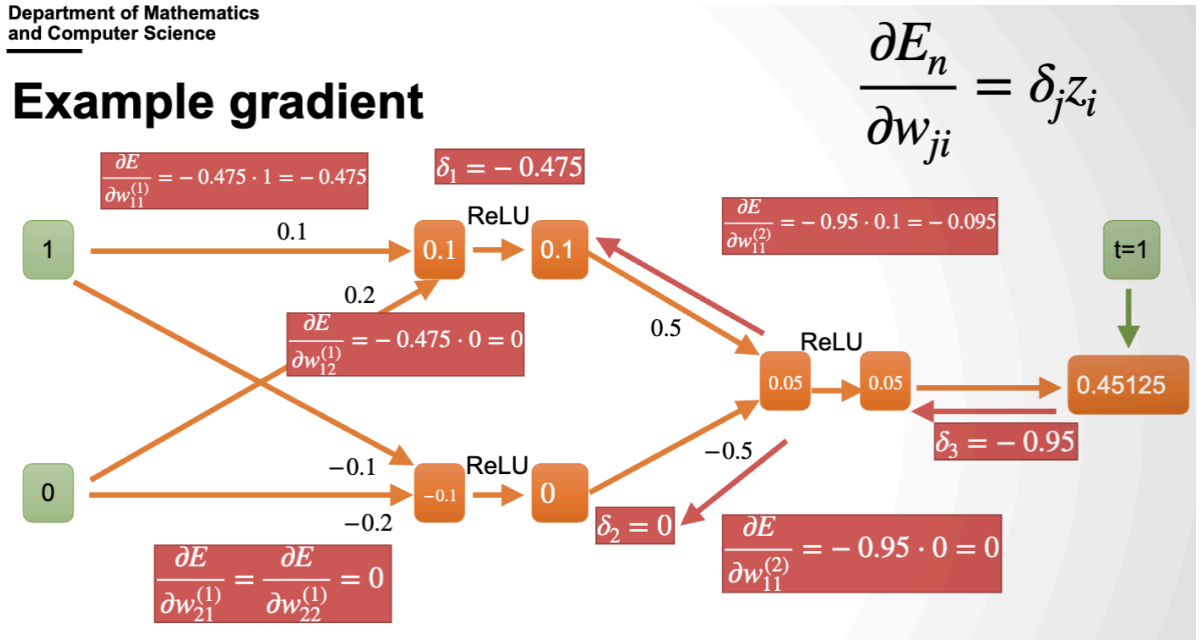

Figure 11: Easy example

The Backprop Algorithm

Input: input vector x_n